



ITE/CSC220 · WEB DEVELOPMENT II · WEEK 1 · SESSION 1

Advanced JavaScript

JS Fundamentals Review • ES6+ Features

Where this course takes you

Wk 1–2

Advanced & Async JavaScript

ES6+, promises, async/await, Fetch

Wk 3

Git & GitHub

Version control & collaboration

Wk 4–5

Node.js & Express

Server-side development

Wk 7–8

MongoDB & REST APIs

Databases & API design

Wk 9

Auth & Security

JWT, hashing, sessions

Wk 10–11

React

Components, hooks, API integration

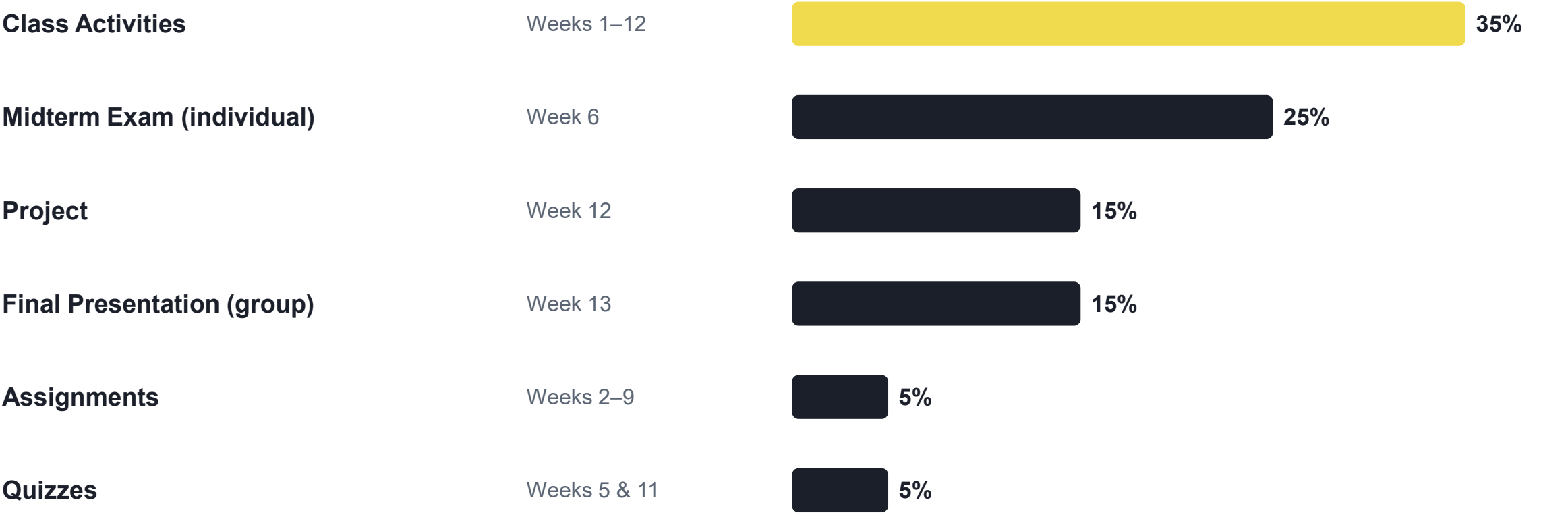
Wk 12–13

Project & Deployment

Build, deploy, present

You are here: Weeks 1–2 sharpen the JavaScript you'll use in every later week of this course.

How you'll be assessed



35% comes from showing up and coding in class. Attendance + participation in weekly coding exercises is your biggest single grade component.

Today's agenda

1

Course & week introduction

Roadmap, assessment, tools check (VS Code, Node.js, DevTools)

2

JavaScript fundamentals review

Variables & scope, types & equality, functions, array methods

3

ES6+ features

Arrow functions, template literals, destructuring, spread/rest, modules

4

Coding exercise

Refactor legacy JavaScript into modern ES6+

Variables & scope: var, let, const

- const by default — use for values that are never reassigned
- let when reassignment is needed (counters, accumulators)
- var is function-scoped and hoisted — avoid in modern code
- let / const are block-scoped: they live only inside { }

Rule of thumb

Start with const. Change to let only when the compiler / linter complains. Never write var in this course.

```
scope.js

// block scope
{
  let a = 1;
  var b = 2;
}

// console.log(a); (ReferenceError)
// console.log(b); // 2

const user = "Nan";
user = "Mai";      // TypeError: assignment
                  // to constant variable

const scores = [80, 92];
scores.push(75); // A const array can still be modified
```

Types, coercion & equality

- Primitives: string, number, boolean, null, undefined, symbol, bigint
- Objects: arrays, functions, dates — everything else
- `==` compares after type coercion — full of surprises
- `===` compares value and type — always use this
- `typeof x` tells you the type (with the famous `typeof null` quirk)

Note: Always use strict comparison: `===` and `!==`.

```
equality.js

"5" == 5      // true   (coerced!)
"5" === 5     // false  (safe)
0 == false    // true
0 === false   // false
null == undefined // true
null === undefined // false
5 !== "5"     // true because their types are different

typeof "hi"    // "string"
typeof 42      // "number"
typeof null    // "object"  <- quirk!
typeof undefined // "undefined"
```

Functions & the array method toolbox

```
functions.js

// declaration (hoisted)
function add(a, b) { return a + b; }

// expression (not hoisted)
const sub = function (a, b) { return a - b; };
```

```
arrays.js

const nums = [1, 2, 3, 4, 5];

nums.map(n => n * 2)           // [2,4,6,8,10]
nums.filter(n => n % 2 === 0) // [2,4]
nums.reduce((sum, n) => sum + n, 0) // 15
nums.forEach(n => console.log(n))
nums.find(n => n > 3)           // 4
nums.includes(3)               // true
```

map

transform every element → new array

filter

keep elements that pass a test

reduce

boil an array down to one value

forEach

run a side effect per element

find / includes

locate or check membership

PART 2

ES6+ Features

The modern syntax you'll use in Node, Express and React:

arrow functions · template literals · destructuring · spread & rest · modules



Arrow functions

```
arrows.js

// classic function expression
const double = function (n) {
  return n * 2;
};

// arrow function
const double2 = (n) => {
  return n * 2;
};

// implicit return (one expression)
const double3 = n => n * 2;

// multiple params need parentheses
const area = (w, h) => w * h;
```

- Shorter syntax — ideal for callbacks: `nums.map(n => n * 2)`
- Implicit return when the body is a single expression (no braces)
- No own `this` — arrows inherit this from the surrounding scope
- Because of that: don't use arrows as object methods that need this (see more details in next slide)

Preview: Express routes are written as arrows:
`app.get("/", (req, res) => res.send("Hi"))`

Object methods & *this*

```
methods.js

// method using this (correct)
const student = {
  name: "Wendy",
  introduce() {
    console.log("Hello, I am " + this.name);
  }
};
student.introduce(); // "Hello, I am Wendy"

// arrow method (avoid when you need this)
const student2 = {
  name: "Wendy",
  introduce: () => {
    console.log(this.name);
  }
};
student2.introduce(); // undefined
```

- A method is a function stored inside an object
- In a normal object method, `this` refers to the object that called it
- Arrow functions do not have their own `this` — they inherit it from the surrounding scope
- So, don't use arrow functions as object methods when you need `this`

Rule of thumb: Use method shorthand or `function` for object methods that access object properties with `this`.

Template literals & destructuring

```
templates.js

const name = "Ploy", score = 92;

// old
console.log("Hi " + name + ": " + score);

// template literal (backticks)
console.log(`Hi ${name}: ${score}`);

// multi-line strings work too
const html = `
  <li>${name} - ${score}</li>
`;
```

```
destructuring.js

const user = { id: 7, name: "Beam",
               role: "admin" };

//Without destructuring, access each property
separately
const name = user.name;
const role = user.role;

// object destructuring
const { name, role } = user;
```

Why it matters: destructuring is everywhere in React — e.g. `const { useState } = React` and props unpacking.

Destructuring: renaming variables

rename.js

```
const user = { id: 7, name: "Beam", role: "admin" };

// get name, but save it as n
const { name: n } = user;

console.log(n);      // "Beam"
console.log(name);   // ReferenceError

// rename more clearly
const { role: userRole } = user;
console.log(userRole); // "admin"
```

- Use renaming when the original property name is too short, unclear, or conflicts with another variable.
- ``name`` is the object property. ``n`` is the new variable name.
- This does not change the original object — it only creates a new variable.

Pattern: `const { propertyName: newVariableName } = object;`

Destructuring: default values

defaults.js

```
const user = { name: "Beam" };

// age does not exist in user
const { age = 18 } = user;
console.log(age); // 18

const user2 = { name: "Mint", age: 25 };
const { age: age2 = 18 } = user2;
console.log(age2); // 25

// default is used only when value is undefined
```

- A default value is used when the property is missing or `undefined`.
- If the property already has a value, JavaScript keeps the real value.
- This helps prevent `undefined` in your output or UI.

Pattern: `const { property = defaultValue } = object;`

Destructuring: arrays by position

arrays.js

```
const scores = [10, 20, 30];

// array destructuring uses position
const [first, second] = scores;
console.log(first);    // 10
console.log(second);   // 20

// skip the second value
const [a, , c] = scores;
console.log(a);        // 10
console.log(c);        // 30
```

- Array destructuring extracts values based on their position, not their names.
- ``first`` gets index 0, ``second`` gets index 1.
- You can skip an item by leaving an empty space between commas.

Compare: objects use property names; arrays use positions.

Destructuring in function parameters

params.js

```
const user = { name: "Beam", role: "admin" };

// old style
const greetOld = user => `Hi ${user.name}`;

// destructure directly in the parameter
const greet = ({ name }) => `Hi ${name}`;

console.log(greet(user)); // "Hi Beam"

// the function receives the object, then extracts name
```

- The function still receives an object as its argument.
- The parameter `{ name }` immediately extracts the `name` property.
- This is very common in React props, such as `function Profile({ name })`.

Pattern: `const fn = ({ property }) => use(property);`

Spread & rest: the three dots ...

spread.js

```
// SPREAD - expand into pieces
const a = [1, 2], b = [3, 4];
const merged = [...a, ...b]; // [1,2,3,4]
const copy = [...a];         // real copy

const user = { name: "Mek", age: 20 };
const updated = { ...user, age: 21 };
// { name: "Mek", age: 21 }
```

rest.js

```
// REST - gather the leftovers
const sum = (...nums) =>
  nums.reduce((t, n) => t + n, 0);

sum(1, 2, 3, 4); // 10

const [head, ...tail] = [1, 2, 3];
// head = 1, tail = [2, 3]
```

Spread

same dots, opposite jobs: spread expands an array/object out

Rest

rest collects many values in — in parameters and destructuring

You'll use it for

copying state immutably in React and merging request data in Express

ES modules: import & export

mathUtils.js

```
// mathUtils.js
export const PI = 3.14159;
export const square = n => n * n;

export default function cube(n) {
  return n * n * n;
}
```

app.js

```
// app.js
import cube, { PI, square } from
  "./mathUtils.js";

square(4); // 16
cube(2);   // 8
```

- One file = one module; each module has its own scope
- Named exports — many per file, imported with { braces }
- Default export — one per file, imported without braces
- In the browser: `<script type="module" src="app.js">`
- Node.js also supports CommonJS (require) — we'll meet it in Week 4

Why it matters: every React file you write in Weeks 10–11 starts with import and ends with export.

Object property shorthand

object-shorthand.js

```
const name = "Fah";
const age = 19;

// old way
const student1 = {
  name: name,
  age: age
};

// shorthand when names match
const student2 = { name, age };

// method shorthand
const student3 = {
  name,
  age,
  greet() {
    return `Hi ${this.name}`;
  }
};
student3.greet(); // "Hi Fah"
```

What it means

- Use { name, age } when variable names match property names.
- It is the same as { name: name, age: age }, but shorter.
- Method shorthand: greet() { ... } is the short version of greet: function() { ... }.
- Useful for cleaner objects, API data, and React props.

Old way

```
name: name
age: age
```

Shorthand

```
name
age
```

Rule of thumb: When property name and variable name are the same, write the name once.

More modern niceties



```
const name = "Fah", age = 19;

// enhanced object literals
const student = { name, age,
  greet() { return `Hi ${this.name}`; }
};

// optional chaining ?.
const city = user.address?.city;
// undefined instead of a crash

// nullish coalescing ??
const limit = input ?? 10;
// 10 only if input is null/undefined
```

- Shorthand properties: { name } instead of { name: name }
- ?. stops safely when something is null / undefined — crucial when reading API data
- ?? provides defaults without treating 0 or "" as missing (unlike ||)
- These appear constantly when handling MongoDB documents and JSON responses

Exercise: modernize this code

```
legacy.js — refactor me

var students = [
  { name: "Anna", score: 82 },
  { name: "Boon", score: 95 },
  { name: "Chai", score: 58 }
];

function getPassed(list) {
  var result = [];
  for (var i = 0; i < list.length; i++) {
    if (list[i].score >= 60) {
      result.push("Passed: " +
        list[i].name + " (" +
        list[i].score + ")");
    }
  }
  return result;
}
```

Your tasks

- Replace var with const / let appropriately
- Rewrite getPassed as an arrow function
- Use filter + map instead of the for loop
- Use destructuring for name and score
- Build the string with a template literal
- Stretch: put it in a module and import it

Target: the whole function should fit in 2–3 lines of modern JavaScript.

What you can do now

- 1 Choose `const` / `let` correctly and explain block scope
- 2 Use `===` and predict coercion pitfalls of `==`
- 3 Write arrow functions with implicit returns
- 4 Destructure objects & arrays; use spread and rest
- 5 Split code into ES modules with `import` / `export`

Next session

DOM manipulation & events — selecting elements, changing the page from code, and reacting to user clicks and input.